

Week 39 exercises – INF201 2026

Working with clinical data

The file `psa_clinical_data.csv` contains mock clinical data. This represents data for a project analysing follow-up data for patients who underwent surgery (radical prostatectomy) for prostate cancer. After the surgery the patients are followed up at regular intervals, with tests checking for levels of prostate-specific antigen (PSA) in the blood. This level should be zero (or at least close to zero) if the patient is cancer-free. An increase in PSA level indicates a cancer recurrence.

The data has the following columns:

- `study_id`: The name of the study
- `patient_id`: Each patient has an id-number
- `date_of_birth`: The patient's date of birth,
- `surgery_date`: Date of the patient's surgery
- `psa_value`: Level of prostate-specific antigen (PSA, ng/mL) based on blood-test
- `psa_date`: Date of the blood-test

Note that this file is a mixture of data from three different studies, indicated by `study_id`. The file has one line per PSA measurement, meaning that it has multiple lines of data per patient.

Task 1 – Organise the data

You have been asked to organize this dataset. You want to split the data into several files and create a folder structure for them. This structure should be as follows:

1. Project root folder: A main directory for the entire project
2. Study folders: One subfolder for each `study_id`
3. Study summary file: Inside each study subfolder, generate a CSV file named `<study_id>_summary.csv` containing one row per patient with the following columns:
 - `unique_id`: A combination of `study_id` and `patient_id`, constructed in a way that allows us to sort the patient by id in a logical manner
 - `date_of_birth`: The patient's date of birth.
 - `surgery_date`: The date of surgery.
 - `follow_up_days`: Total days passed between `surgery_date` and the most recent `psa_date`.

- number_of_psa_measurements: Total count of available PSA blood tests.
 - patient_folder_path: Relative path to the patient's individual subfolder.
4. Patient subfolders and files:
- Inside each study folder, create a subfolder for each patient named using their unique identifier.
 - Inside this folder, save a CSV file containing all follow-up records for that patient, sorted chronologically by psa_date.

Some things to keep in mind here:

- What should happen with existing files and folders if your script/function is run several times? How do you handle this situation?
- Within each of the three studies, the patients have been given a patient_id from 1 and up to the max number of patients in the study. How can you give the files and folder names to avoid confusing patient 1 from study A12 with patient 1 from study C08?
- How do you ensure that files and folders can be sorted in a logical manner?
- The datetime module has useful functions for working with dates and time intervals.
- The script(s) you write in this task will generate almost 200 folders and files on your computer. If that is a bit much, you may use a smaller subset of the original csv-file to work with.

Task 2 – Include data from large files

Now you want to include some gene sequencing data that is available for a subset of the patients. Unfortunately, these files have not been given very informative file names, so you need to check the header of each file to find the patient and study ids. The files are also very large, typically several gigabytes. Assume that the beginning of each file has a header, where each line of the header starts with the characters ##. Each line looks something like this:

```
##fileformat=VCFv0.madeup
##fileDate=20260916
##study_id=A12
##patient_id=132
##other_info=something
```

Write a function that iterates through the folder of sequencing files does the following:

1. Reads each file line-by-line to extract `study_id` and `patient_id` from the header.
2. Stops reading the file immediately once the header lines end (as soon as a line does not start with `##`).
3. Generates a summary csv-file named `sequencing_manifest.csv` saved in the main project folder containing the following columns:
 - `unique_id` (the same you generated in task 1)
 - `study_id`
 - `patient_id`
 - `sequencing_file_path` (relative path to the raw file, for example “`raw_sequencing_data/seq_001.vcf`”)

Hand-in information

- **Do not** include all the generated files in your submission! We *only* want the script(s).
- Remember to include filename extension for all submitted files
- You may submit Python scripts (.py) or Jupyter notebooks (.ipynb)
- If you submit a Jupyter notebook, please upload a pdf version as well
- AI chat-log can be submitted in its entirety or summarized (see `lecture_02`)